

AGENT DEBUGGING

I kept asking the same question after every agent failure.

Not "what did the agent do?"

The logs already show that.

But "why did it do that?" and "which step broke it?"

Those took hours to untangle manually.

So I built an MCP tool that answers both in seconds.

HOW IT WORKS

Pass in any agent trace JSON. Get root causes back.

No SDK changes. No instrumentation. Works with LangChain, OpenAI Agents SDK, AutoGen, Claude SDK, Microsoft Agent Framework, or anything that produces a step log.

judge_trace

Root cause analysis, 4-dimension score, grade A to F, one concrete fix.

trace_breakdown

Scores every step individually. Flags REDUNDANT_TOOL_CALL, REASONING_GAP, GOAL_DRIFT, PREMATURE_STOP.

efficiency_score

Free, no API key

Token, latency, and redundancy scoring. Deterministic. Runs instantly. Safe for CI/CD.

WHEN THE AGENT GETS IT RIGHT

Customer support agent. Refund processed correctly.

Most teams assume their agents work. This is how you confirm it.

TRACE: 5 STEPS

lookup_order

process_refund

Correct output

A

0.96 / 1.0

PRODUCTION-READY

Goal completion	1.00
Tool usage	0.95
Output quality	0.95
Reasoning clarity	0.95

Root causes: none | Redundant calls: 0 | Failed calls: 0

NO ERRORS. ALL TOOLS RETURNED OK. STILL WRONG.

Financial report agent. CFO got an incomplete summary.

Goal: summarise Q3 performance and flag anomalies. Every tool call succeeded. The output looked professional. The -23% APAC revenue drop was never mentioned.

FETCH_REPORT

OK

PARSE_DATA

OK

SEND_TO_CFO

OK

ctrl+F finds nothing. Grep finds nothing. Looks fine in every log.

JUDGE_TRACE RESULT

C 0.55 / 1.0 Needs optimisation

Root cause: Agent summarised top-level numbers correctly but did not perform the anomaly detection step it was explicitly asked for. The APAC drop was present in the parsed data and never surfaced.

Fix: Add a dedicated anomaly scan step after parse_data. Do not generate the summary until all flagged conditions are checked.

WASTE YOU CAN ONLY SEE IN THE TRACE

Data pipeline agent queried the same database 3 times.

Identical inputs. Identical outputs. Then sent an email with half the data missing.

REDUNDANT DB CALLS

2

of 3 total identical

WASTED LATENCY

13.6s

on duplicate queries

EFFICIENCY SCORE

0.36

out of 1.0

JUDGE_TRACE RESULT

D 0.60 / 1.0 Needs optimisation

Root causes: Steps 3, 4, and 5 are identical `query_sales_db` calls. Email body sent to leadership omitted top products entirely.

Fix: Add tool-call deduplication before dispatch. Implement a report formatting step that uses all retrieved data before sending.

WHERE IT FITS IN YOUR STACK

Your observability stack tracks what happened. This explains why.

LangSmith, Arize Phoenix, W&B Weave give you history across many runs.

This answers a narrower question: **why did this specific trace fail, and what exactly needs to change?**

Works in

Claude Desktop
Cursor
VS Code (Copilot MCP)
Any studio MCP client

Model support

Claude Opus 4.6, GPT-5.4, GPT-4o
GPT-4o mini, Claude Haiku, Sonnet
Any LiteLLM-supported model

OPEN SOURCE / MIT

agent-trace-intelligence

Find the failure. Pinpoint the step. Fix the root cause.

Install `pip install agent-trace-intelligence`

Repo `github.com/harinarayn/agent-trace-intelligence`

CI/CD: run `efficiency_score` on every agent invocation for free. Use `judge_trace` on failures and regressions. Under \$0.01 per trace with gpt-4o-mini.

Evaluated using [Claude Opus 4.6](#) via Microsoft Foundry. Configure your own model via environment variables: OpenAI, Azure, Anthropic, or any LiteLLM-supported provider.

Adapters included for LangChain, OpenAI Agents SDK, AutoGen, and Microsoft Agent Framework GA 1.0.

PRs and feedback welcome.